

ControlPlane.ai

Project README and usage guide

ControlPlane.ai is a policy-driven governance layer for AI-generated responses. It evaluates text through multiple checks, combines the results into a confidence score, and returns an allow, flag, or block decision.

The project includes:

- A browser dashboard for interactive chat, evaluation, and PDF-based auditing.
- A FastAPI backend for integrating governance into another application.
- A batch CLI for evaluating simulated responses.
- Rule-based PII detection for email addresses and phone numbers.
- Local knowledge-base retrieval and verification.
- Optional Groq-powered response generation and AI-as-a-judge evaluation.
- Configurable per-use-case policy thresholds.
- JSON feedback logging for flagged results.

Live demo: control-plane-ai.onrender.com

How the system works

```
User prompt / AI response
|
v
+-----+
| Heuristics          | PII and fast rule checks
| RAG verifier        | Local knowledge-base matching
| AI judge            | Optional Groq review
+-----+
|
v
Confidence scoring
|
v
Policy decision: allow / flag / block
|
v
Feedback log for non-allowed results
```

The checks run independently. Each tier returns pass, fail, or not_applicable with a score from 0 to 1 and a reason. Scores are averaged across applicable tiers. If one or more tiers are not applicable, the result is marked as incomplete and capped at 0.75; if every tier is not applicable, the confidence is 0.0.

This is a governance prototype, not a guarantee of factual correctness. The AI judge is another model call rather than a source of truth, and RAG verification only works when a matching fact exists in the local knowledge base.

Requirements

- Python 3.9 or newer

- A virtual environment is recommended
- A Groq API key is optional for local heuristics/RAG operation, but required for live LLM generation and the AI judge

Installation

Windows PowerShell

```
python -m venv .venv
.\\.venv\\Scripts\\Activate.ps1
python -m pip install -r requirements.txt
```

If PowerShell blocks activation, run this in a PowerShell session where you have permission to change the execution policy:

```
Set-ExecutionPolicy -Scope CurrentUser RemoteSigned
```

macOS / Linux

```
python3 -m venv .venv
source .venv/bin/activate
python -m pip install -r requirements.txt
```

Configuration

Create a .env file in the project root for Groq features:

```
GROQ_API_KEY=your_groq_api_key
GROQ_CHAT_MODEL=openai/gpt-oss-20b
GROQ_JUDGE_MODEL=llama3-8b-8192
```

.env is ignored by Git. If GROQ_API_KEY is missing, the application still starts:

- The heuristics tier continues to work.
- Local knowledge-base fallback responses can still be returned.
- The judge returns not_applicable.
- LLM-generated answers are replaced by a deterministic fallback or an explanatory message.

Policy thresholds are configured in config.yaml. The available profiles are:

- customer_support_chatbot - medium strictness
- internal_knowledge_assistant - high strictness
- decision_support_regulated - strictest profile

Each profile defines thresholds for heuristics, rag, and judge, plus the actions used for passing, review, and critical results. Unknown use cases use a fallback based on whether any tier failed.

The active runtime policy loader uses config.yaml. config/policies.yaml contains an earlier, more granular policy format and is retained as a reference; it is not the file used by the current FastAPI evaluation path.

Start the web application

From the project root, with the virtual environment activated:

```
uvicorn main:app --reload --host 127.0.0.1 --port 8000
```

Then open <http://127.0.0.1:8000>. FastAPI's interactive API documentation is available at <http://127.0.0.1:8000/docs>.

The equivalent Python entry point is:

```
python main.py
```

Dashboard features

- **Evaluate:** submit an existing AI response and inspect tier results, confidence, and policy action.
- **Chat:** ask a question, retrieve matching internal facts, generate a response, and view its governance result.
- **PDF audit:** upload a text-based PDF. The server keeps one active PDF in process memory until another PDF is uploaded. Prompts then produce an unconstrained answer, a document-grounded answer, and an accuracy audit of the unconstrained answer against the PDF.

Scanned/image-only PDFs are not supported unless they contain an extractable text layer.

API usage

Evaluate an existing response

POST /api/evaluate

Request body:

```
{
  "user_prompt": "Summarize this document.",
  "ai_response": "Contact me at jane@example.com or call 555-123-4567.",
  "use_case": "customer_support_chatbot"
}
```

Example with curl:

```
curl -X POST http://127.0.0.1:8000/api/evaluate \
-H "Content-Type: application/json" \
-d '{"user_prompt":"Summarize this document.","ai_response":"Contact me at jane@example.com","use_case":"customer_support_chatbot"}'
```

The response contains the original request, each tier's status, score, and reason, the overall_confidence, and the final decision.

Chat with governance

POST /api/chat

```
{
  "message": "What is the remote work stipend?",
  "use_case": "internal_knowledge_assistant",
  "policy_path": "config.yaml",
  "retrieval_limit": 5
}
```

policy_path, retrieval_limit, and context are optional. The response includes reply, a separately generated rag_answer, retrieved facts, tier results, scoring, and the policy decision.

Upload a PDF for auditing

POST /api/upload-pdf with a multipart form field named file:

```
curl -X POST http://127.0.0.1:8000/api/upload-pdf \
-F "file=@./path/to/reference.pdf"
```

After a successful upload, call /api/chat or /api/evaluate. While a PDF is active, both routes use the PDF audit flow. Uploading another PDF replaces the current context. The active PDF is process-local and is not persisted to disk or shared between multiple server workers.

Batch CLI

The CLI expects a JSON array. Each item can contain a context object and a response.text value:

```
[
  {
    "context": {
      "use_case": "internal_knowledge_assistant",
      "region": "US"
    },
    "response": {
      "text": "The response to evaluate goes here."
    }
  }
]
```

Run the included sample:

```
python -m src.main --input data/simulated/example_responses.json
```

Override the use case for every item:

```
python -m src.main --input data/simulated/example_responses.json --use-case
decision_support_regulated
```

The CLI prints one JSON result per item. Direct calls through orchestrator.evaluate_request append non-allowed results to feedback_log.json; the chat/CLI feedback hook is currently a placeholder.

Knowledge base

The local enterprise facts used by retrieval and RAG verification are stored in src/data/knowledge_base.json. Add or update entries as JSON key/value pairs, then restart the server or CLI process so the cached knowledge base is reloaded.

The sample facts cover remote-work stipends, password and MFA rules, and the Mess Management System deployment and release process.

Project structure

```
.
+-- main.py           FastAPI application and browser dashboard
+-- orchestrator.py   Parallel evaluation orchestration
+-- config.yaml       Active policy profiles and thresholds
+-- requirements.txt   Python dependencies
+-- feedback_log.json  Logged non-allowed evaluations
+-- src/
|   +-- chat_pipeline.py  Retrieval, generation, and chat governance
|   +-- pdf_audit.py      PDF context, grounded generation, and PDF audit
|   +-- models/schemas.py Pydantic request/result models
|   +-- tiers/           Heuristics, RAG, and AI-judge checks
|   +-- engine/          Groq client, scoring, and policy decisions
|   +-- data/knowledge_base.json Local enterprise knowledge base
|   +-- main.py          Batch CLI entry point
+-- data/simulated/    Sample batch input
+-- config/            Legacy/reference policy format
+-- tests/             Automated tests
+-- docs/              Design decisions and project notes
```

Testing

Run the test suite from the repository root:

```
python -m pytest
```

The PDF audit tests mock the Groq client, so they do not require network access or a real API key.

Development notes and limitations

- The feedback log is JSON and is written to the working directory. Concurrent writers are not coordinated.
- PDF context is held in memory and there is only one active document per process.
- PDF text is truncated before grounded generation to stay within the model context budget.
- The local RAG verifier is keyword/topic based; it is not a vector database or a general-purpose semantic retrieval system.
- AI-as-a-judge results can be wrong and should be treated as a review signal.
- Automatic rewriting/redaction of flagged output is intentionally out of scope.
- Feedback logging supports later offline threshold tuning; it does not automatically retrain a model.

See docs/decisions.md for the design rationale behind coverage tracking, AI-as-judge limitations, and the feedback loop.

License

No license file is currently included. Add a license before distributing the project outside its intended challenge or internal-use context.